Министерство науки и высшего образования Российской Федерации
Национальный исследовательский университет ИТМО
Мегафакультет компьютерных технологий и управления
Факультет программной инженерии и компьютерной техники

Отчет по лабораторной работе № 2

по курсу «Механизмы надежности и безопасности вычислительных систем»

Тема: «Моделирование и верификация в среде SPIN»

Выполнил студент группы P4219 _____ Д. Л. Симоновский
(подпись)

Руководитель _____ В. Ю. Пинкевич
(подпись)

28 октября 2026 г.

Санкт-Петербург
2026

Содержание

1 Задача **2**

2 Ход работы **2**
2.1 Описание моделируемой системы 2
2.2 Взаимодействие процессов и принятые предположения 3
2.3 Реализация модели на языке Promela 4
2.4 Описание верифицируемых свойств 12
2.5 Автоматизация проверки 19
2.6 Верификация модели 22

3 Выводы **23**

Список использованных источников **24**

1. Задача

В рамках лабораторной работы требуется получить практические навыки моделирования взаимодействующих процессов на языке Promela и проверки корректности их работы с использованием верификатора SPIN. Для этого необходимо выбрать некоторую систему и выполнить верификацию алгоритма ее функционирования.

Для выбранной системы требуется составить текстовое описание, позволяющее понять состав участников, порядок их взаимодействия и правила обработки событий. При необходимости описание следует дополнить схемами взаимодействия процессов и алгоритмов их работы. На основе этого описания необходимо построить модель на языке Promela и пояснить принятые при моделировании предположения.

Далее требуется сформулировать не менее десяти свойств системы и представить их в виде LTL-формул. Если для проверки свойства недостаточно исходных переменных модели, необходимо предусмотреть дополнительные средства наблюдения за ее состоянием. Подготовленные свойства следует проверить в среде SPIN, после чего проанализировать результаты, описать выявленные ошибки или подтвердить соответствие модели сформулированным требованиям.

В заключении необходимо указать, какие предположения оказались существенными для верификации, потребовалось ли уточнять модель или свойства и в каких границах полученные результаты характеризуют работу выбранной системы.

2. Ход работы

2.1. Описание моделируемой системы

В качестве системы для верификации выбран обмен запросами и ответами по протоколу Modbus. В рамках работы рассматривается небольшая сеть, состоящая из одного ведущего устройства Master и трех ведомых устройств Slave с адресами 1, 2 и 3. Master формирует запрос, все Slave получают его копию, после чего каждый из них определяет, требуется ли обрабатывать этот запрос.

Основной операцией является запись одного регистра с кодом функции `0x06`. У каждого Slave предусмотрены два регистра с адресами 0 и 1. Если запрос содержит адрес конкретного устройства, запись выполняет только это устройство. После успешной записи оно возвращает ответ, повторяющий код функции, адрес регистра и записанное значение. Такой порядок обработки соответствует назначению функции Write Single Register [2].

Кроме адресной записи, модель предусматривает широковещательный запрос с адресом 0. Его обрабатывают все три Slave, однако ответ Master не отправляется. Для проверки отсутствующего получателя используется адрес 4: такого устройства в сети нет, поэтому запрос остается без ответа.

Перед записью Slave проверяет адрес получателя, CRC, код функции и адрес регистра. Кадр с неверной CRC игнорируется. При верной CRC неподдерживаемая функция вызывает исключение с кодом 1. Если функция поддерживается, но указан несуществующий регистр, возвращается код 2. Исключения отправляются только в ответ на адресные запросы. На широковещательный запрос Slave не отвечает, даже если в нем есть ошибка.

Таблица 2.1. Основные параметры модели

Параметр	Принятое значение
Состав участников	Один Master и три Slave
Адрес получателя	0 - всем устройствам 1-3 - конкретному Slave 4 - отсутствующему устройству
Адрес регистра	0 или 1 2 - ошибочный адрес
Значение для записи	1 или 2 Начальные значения регистров равны 0
Код функции	6 - запись одного регистра 3 - неподдерживаемая функция
Корректность CRC	Логический признак `crc_ok`
Число активных запросов	Одна транзакция за раз

Master повторяет обмен в бесконечном цикле. Параметры очередного запроса выбираются недетерминированно, поэтому верификатор рассматривает различные запросы и допустимые порядки выполнения процессов. Адрес получателя, регистр, значение, функция и корректность CRC выбираются независимо. Поэтому запрос может быть корректным или содержать сразу несколько ошибок. Например, Master может отправить функцию `FC03` с адресом регистра 2 и неверной CRC. Всего предусмотрено 120 сочетаний параметров запроса: пять адресов получателя, три адреса регистра, два значения, две функции и два состояния CRC.

2.2. Взаимодействие процессов и принятые предположения

На рисунке 2.1 приведен пример адресного обмена со Slave 2. Общий запрос доставляется каждому устройству через отдельный канал. Slave 1 и Slave 3 проверяют адрес и завершают обработку без изменения регистров. Slave 2 выполняет запись и помещает ответ в общий канал ответов. Уведомление о завершении отправляет каждый из трех обработчиков.

Каналы запросов имеют емкость в одно сообщение. Это позволяет каждому Slave независимо получить свою копию одного и того же кадра. Общий канал ответов также рассчитан на одно сообщение, поскольку адресным получателем может быть только одно устройство. Служебный канал завершения вмещает три уведомления. Эти уведомления нужны для согласования шагов модели и не являются сообщениями протокола Modbus.

Результат транзакции фиксируется после получения всех трех уведомлений. При адресном запросе наличие сообщения в канале ответов означает получение ответа. Если канал пуст, Master фиксирует тайм-аут. При широковещательном запросе Master сразу фиксирует завершение без ответа. Таким образом, тайм-аут описывает логический результат обмена; числовой интервал ожидания в модели не задается.

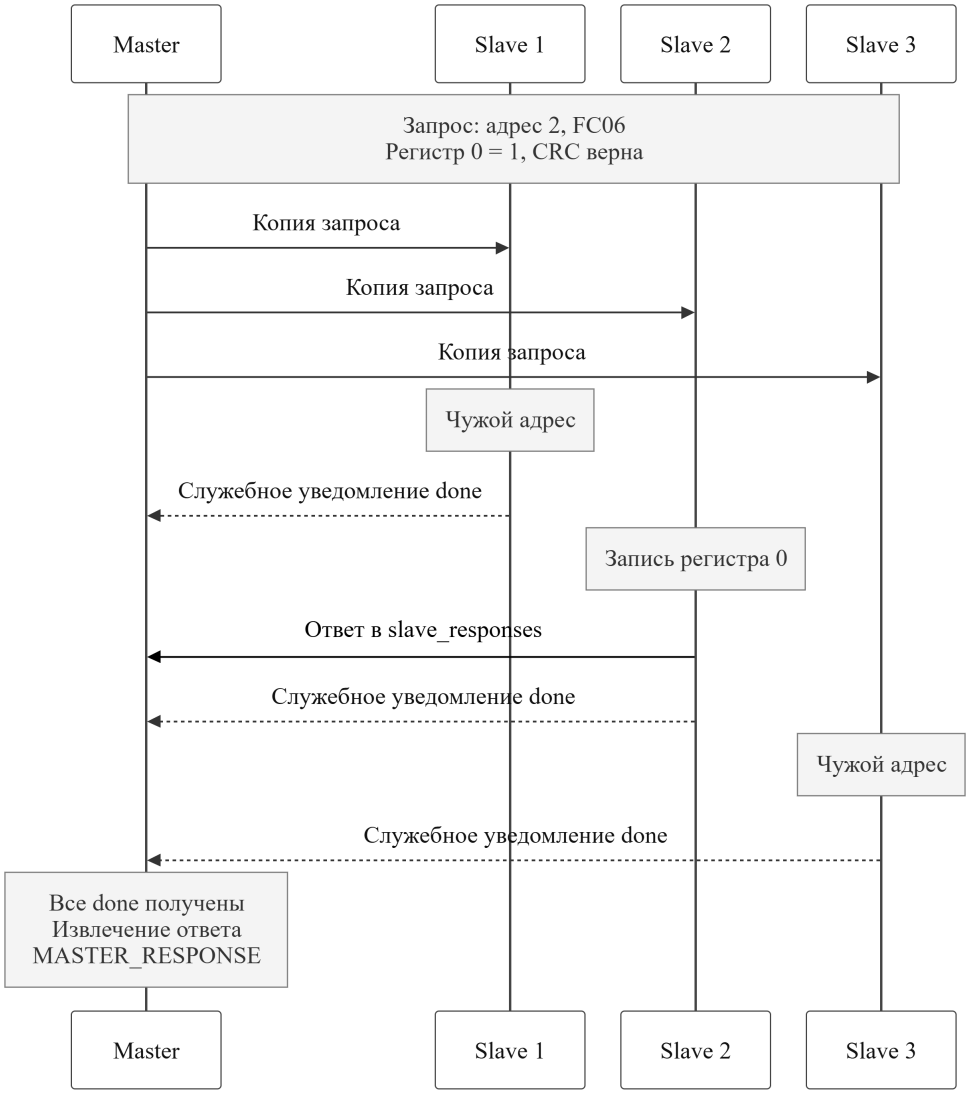


Рис. 2.1. Один из порядков взаимодействия процессов при запросе к Slave 2

Передача кадра представлена доставкой полей сообщения. Корректность CRC задается логическим признаком. Это позволяет сосредоточиться на выборе получателя, условиях записи, содержимом ответа и завершении транзакции. Значения регистров хранятся в переменных типа `byte`, а генератор выбирает только 1 и 2. Проверка показывает, как система обрабатывает запросы с выбранными значениями параметров.

Верифицируемые свойства должны показать, что корректные запросы выполняют нужную запись, ошибочные запросы сохраняют память, посторонние устройства и регистры не изменяются, ответы формируются правильно и начатый обмен обязательно достигает результата. Для этого далее рассматривается реализация модели и средства наблюдения за каждой транзакцией.

2.3. Реализация модели на языке Promela

Модель системы находится в файле `modbus.pml`. В исходном файле приведены комментарии к переменным и основным этапам обмена; полный код доступен в репозитории проекта [1]. Далее рассматриваются функциональные части реализации. В

листингах приведены фрагменты исходного кода без комментариев, а из обработчиков также убрана часть сообщений `printf`, предназначенных для просмотра обмена.

2.3.1. Состояние системы и запуск процессов

Для начала необходимо определить, какую информацию модель должна сохранять между шагами. У Master есть состояние текущего обмена и поля запроса. Память Slave представлена двумя массивами: индекс соответствует адресу устройства. Нулевой элемент массивов не используется, поэтому обращение к памяти Slave 2 записывается непосредственно через индекс 2.

Листинг 2.1. Состояние Master и память устройств

```
#define FC06 6
#define FC03 3
mtype = {
    MASTER_PREPARING, MASTER_WAITING, MASTER_RESPONSE,
    MASTER_TIMEOUT, MASTER_BROADCAST_DONE
};
mtype master_state = MASTER_PREPARING;

byte slave_address;
byte function_code;
byte register_address;
byte register_value;
bool crc_ok;
byte holding_register_0[4];
byte holding_register_1[4];
bool slave_write_performed[4];
```

Все числовые поля объявлены как `byte`. При создании модели их начальное значение равно нулю. После завершения транзакции содержимое регистров сохраняется, поэтому следующие запросы работают с результатами предыдущих записей. Флаг `slave_write_performed` показывает именно факт выполнения операции: повторная запись уже хранящегося значения также должна считаться записью.

Взаимодействие процессов организовано через каналы. Сообщение запроса содержит адрес получателя, код функции, адрес регистра, значение и признак CRC. В ответе последнее поле заменено кодом исключения. Нулевой код исключения означает нормальный ответ. Процесс `init` создает три экземпляра одного обработчика `Slave` и один экземпляр `Master`.

Листинг 2.2. Каналы обмена и начальный запуск

```
chan slave_requests[3] = [1] of { byte, byte, byte, byte, bool };
chan slave_responses = [1] of { byte, byte, byte, byte, byte };
chan slave_done = [3] of { bool };

init
{
    atomic {
        run Slave(1);
        run Slave(2);
        run Slave(3);
        run Master();
    }
}
```

Отдельные каналы запросов позволяют смоделировать получение общего кадра всеми устройствами. Если бы все Slave читали один обычный канал Promela, сообщение получил бы только один из них. В выбранной реализации адрес проверяет сам обработчик, а не механизм доставки.

2.3.2. Подготовка очередной транзакции

Работа Master представляет собой бесконечное повторение одного обмена. Алгоритм приведен на рисунке 2.2. В начале Master находится в состоянии `MASTER_PREPARING`, в котором выполняется подготовка к очередной транзакции. В это состояние он возвращается после каждого обмена. Затем Master очищает ответ и наблюдательные признаки предыдущего обмена, сохраняет значения регистров и выбирает новый запрос.

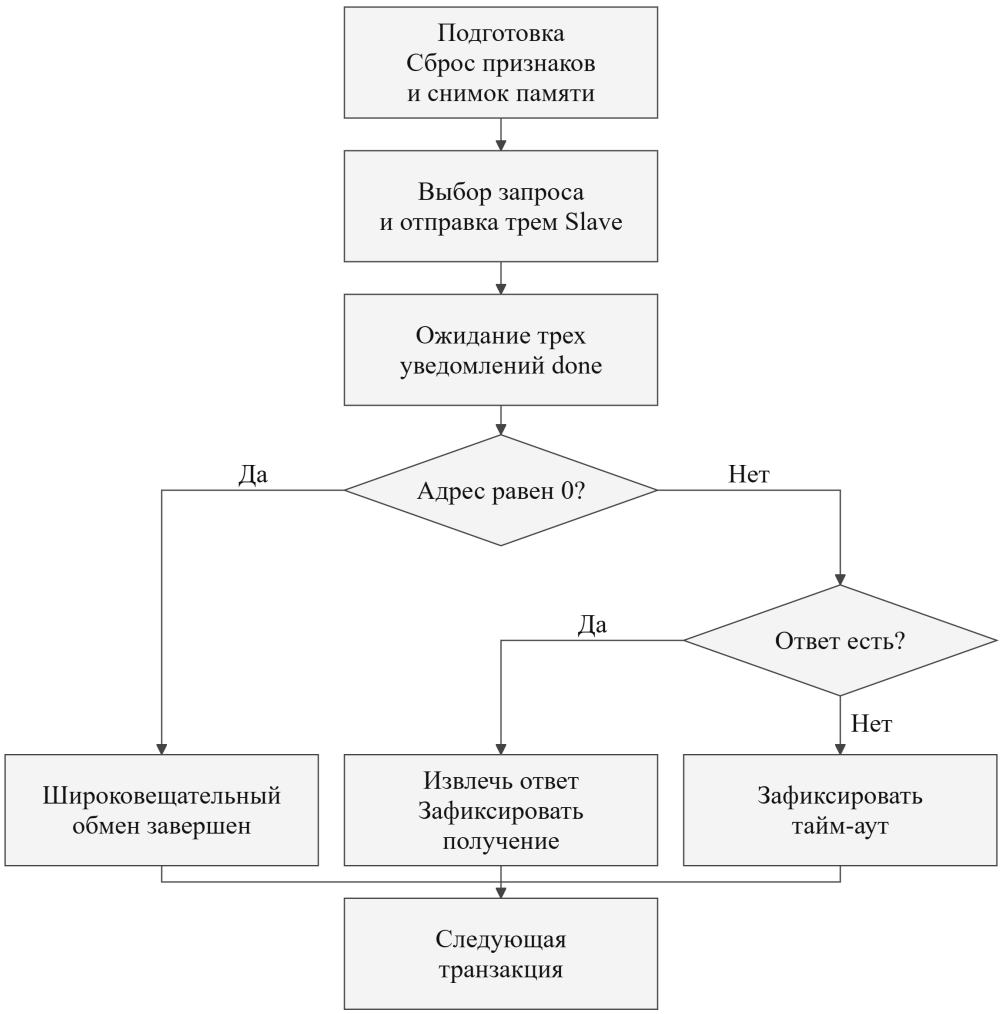


Рис. 2.2. Алгоритм работы процесса Master

Снимок памяти необходим для последующего сравнения. Проверка «не было записи» и проверка «значение не изменилось» имеют разный смысл: запись того же значения не меняет память, но операция при этом выполняется. Поэтому используются и предыдущие значения, и отдельные флаги.

Листинг 2.3. Сброс признаков обмена и сохранение памяти

```
response_slave_address = 0;
response_function_code = 0;
response_register_address = 0;
response_register_value = 0;
response_exception_code = 0;
slave_write_performed[1] = false;
slave_write_performed[2] = false;
slave_write_performed[3] = false;
slave_processed[1] = false;
slave_processed[2] = false;
slave_processed[3] = false;
responses_sent = 0;

previous_register_0[1] = holding_register_0[1];
previous_register_0[2] = holding_register_0[2];
previous_register_0[3] = holding_register_0[3];
previous_register_1[1] = holding_register_1[1];
previous_register_1[2] = holding_register_1[2];
previous_register_1[3] = holding_register_1[3];
```

Далее Master выбирает параметры запроса. В конструкции `if` все ветви с присваиваниями допустимы, поэтому выбор является недетерминированным. Это задает множество вариантов поведения, которое исследует SPIN.

Адрес получателя выбирается из значений от 0 до 4, адрес регистра из значений 0, 1 и 2, а значение для записи равно 1 или 2. Функция выбирается из `FC06` и `FC03`, а признак CRC может быть истинным или ложным. Эти выборы не зависят друг от друга, поэтому модель рассматривает в том числе одновременную ошибку CRC, функции и адреса регистра.

Листинг 2.4. Независимый выбор параметров запроса

```
if
:: slave_address = 0
:: slave_address = 1
:: slave_address = 2
:: slave_address = 3
:: slave_address = 4
fi;
if
:: register_address = 0
:: register_address = 1
:: register_address = 2
fi;
if
:: register_value = 1
:: register_value = 2
fi;
if
:: function_code = FC06
:: function_code = FC03
fi;
if
:: crc_ok = true
:: crc_ok = false
fi;
```

Код FC03 соответствует функции чтения регистров в Modbus, но в рассматриваемой реализации эта функция специально не поддерживается. Здесь она служит примером запроса, на который требуется вернуть исключение. Проверка тем самым относится к устройству, реализующему только FC06.

После подготовки Master переходит в MASTER_WAITING и помещает одинаковые поля в три канала. Сброс признаков, выбор параметров и отправка объединены одним блоком atomic. Он сохраняет исключительное выполнение последовательности, пока ее операторы не блокируются [3]. К началу очередного обмена каналы запросов уже освобождены обработчиками.

Листинг 2.5. Передача копий одного запроса

```
master_state = MASTER_WAITING;
slave_requests[0]!slave_address, function_code,
    register_address, register_value, crc_ok;
slave_requests[1]!slave_address, function_code,
    register_address, register_value, crc_ok;
slave_requests[2]!slave_address, function_code,
    register_address, register_value, crc_ok;
```

2.3.3. Проверки на стороне Slave и выполнение записи

Каждый Slave работает в собственном бесконечном цикле и читает только свой канал slave_requests[slave_id - 1]. Поля принятого кадра сохраняются в локальных переменных. Поэтому один и тот же тип процесса можно использовать для всех трех устройств, передавая ему собственный адрес в параметре slave_id.

Алгоритм обработки показан на рисунке 2.3. Сначала проверяется, адресован ли запрос этому устройству или всем сразу. Затем проверяется CRC. Только после этого определяется допустимость функции и адреса регистра. Такой порядок позволяет игнорировать поврежденный кадр до выполнения операции и формирования ответа.

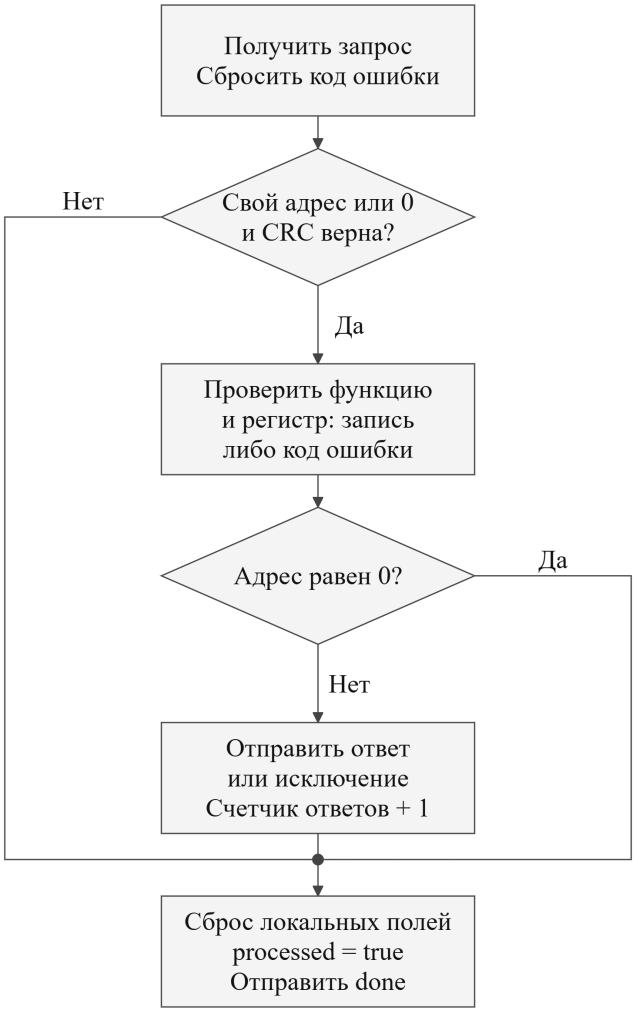


Рис. 2.3. Алгоритм обработки запроса одним Slave

В начале обработки `exception_code` равен нулю. При неподдерживаемой функции он получает значение 1, при ошибочном адресе регистра получает значение 2. Если обе проверки пройдены, Slave записывает выбранный регистр и устанавливает `slave_write_performed[slave_id]`. Эти действия объединены в `atomic`. Другие процессы не могут выполнить свой шаг между изменением регистра и установкой флага. К моменту завершения блока регистр содержит записанное значение, а флаг выполнения записи уже равен `true`.

Листинг 2.6. Проверка запроса, запись и формирование ответа

```
if
:: (request_slave_address == slave_id || request_slave_address == 0) ->
    if
    :: request_crc_ok ->
        if
        :: (request_function_code != FC06) -> exception_code = 1
        :: else ->
            if
            :: (request_register_address > 1) -> exception_code = 2
            :: else ->
                atomic {
                    if
                    :: (request_register_address == 0) ->
                        holding_register_0[slave_id] = request_register_value
                    :: else ->
                        holding_register_1[slave_id] = request_register_value
                    fi;
                    slave_write_performed[slave_id] = true;
                }
            fi
        fi;
        if
        :: (request_slave_address == 0) -> skip
        :: else ->
            if
            :: (exception_code == 0) ->
                atomic {
                    slave_responses!slave_id, request_function_code,
                        request_register_address, request_register_value, 0;
                    responses_sent++;
                }
            :: else ->
                atomic {
                    slave_responses!slave_id, (request_function_code + 128),
                        0, 0, exception_code;
                    responses_sent++;
                }
            fi
        fi
    :: else -> skip
    fi
:: else -> skip
fi;
```

При неверной CRC запрос игнорируется независимо от остальных полей. При верной CRC неподдерживаемая функция дает исключение 1, даже если адрес регистра также ошибочен. Исключение 2 формируется только для функции `FC06` с несуществующим регистром.

Вторая часть этого фрагмента определяет ответ. Для широковещательного запроса выполняется `skip`: память может измениться, но сообщения в канале ответов не возникает. При адресной успешной записи возвращаются исходные поля запроса и нулевой код исключения.

Если обнаружена ошибка функции или регистра, к коду функции прибавляется 128, то есть устанавливается старший бит. В канале передаются также нулевые значения

полей регистра и данных и код исключения. Эти нули относятся к фиксированной структуре сообщения модели: в ней нормальный ответ и исключение используют один тип канала.

Отправка ответа и увеличение `responses_sent` также выполняются совместно. Счетчик сохраняет факт отправки даже после того, как Master извлек сообщение из канала. Поэтому свойство «не более одного ответа» можно проверять на протяжении всей транзакции.

2.3.4. Согласование завершения обмена

После обработки запроса Slave устанавливает признак `slave_processed[slave_id]`. Далее очищаются локальные поля и отправляется служебное уведомление. Очистка локальных переменных не меняет память устройства, но устраняет из состояния завершившего обработку процесса данные старого кадра.

Листинг 2.7. Завершение обработки одного Slave

```
atomic {
    slave_processed[slave_id] = true;
    request_slave_address = 0;
    request_function_code = 0;
    request_register_address = 0;
    request_register_value = 0;
    request_crc_ok = false;
    exception_code = 0;
    slave_done!true;
}
```

Master читает три уведомления, после чего выбирает итоговое состояние. При адресе 0 это `MASTER_BROADCAST_DONE`. В остальных случаях проверяется наличие ответа: он извлекается в глобальные поля и фиксируется `MASTER_RESPONSE`, либо выбирается `MASTER_TIMEOUT`.

Листинг 2.8. Получение результата транзакции

```
slave_done?true;
slave_done?true;
slave_done?true;

if
:: (slave_address == 0) ->
    master_state = MASTER_BROADCAST_DONE;
:: else ->
    if
    :: (len(slave_responses) > 0) ->
        slave_responses?response_slave_address, response_function_code,
            response_register_address, response_register_value,
            response_exception_code;
        master_state = MASTER_RESPONSE;
    :: else ->
        master_state = MASTER_TIMEOUT;
    fi
fi;
```

Ожидание трех уведомлений существенно для проверки свойств. Оно позволяет отличить отсутствие ответа от ситуации, когда нужный Slave еще не получил воз-

можность выполнить свой шаг. Кроме того, при переходе к следующей транзакции все обработчики уже закончили предыдущую.

Для наблюдения за этими условиями дополнительно объявлены снимки памяти, признаки завершения и счетчик ответов. Они используются в свойствах системы, которые далее вынесены в отдельный файл.

Листинг 2.9. Данные для наблюдения за транзакцией

```
byte previous_register_0[4];
byte previous_register_1[4];
bool slave_processed[4];
byte responses_sent;
```

2.4. Описание верифицируемых свойств

Свойства вынесены в файл `properties.pml`, который подключает модель командой `#include "modbus.pml"`. Всего сформулировано 14 LTL-свойств. Они проверяют обработку корректных и ошибочных запросов, сохранность памяти, ответы устройств и завершение обмена.

В формулах используются оператор `[]` со смыслом «всегда», импликация `->` и логические связки. Для свойства прогресса дополнительно используется сильный оператор `U`. Формулы записаны как требования к поведению модели. При генерации верификатора SPIN автоматически строит по отрицанию выбранной формулы автомат `never claim`, который ищет нарушающее ее выполнение [4]. Поэтому описывать `never claim` вручную не требуется. Отдельные `assert`-выражения также не используются, поскольку все выбранные требования выражены в LTL.

2.4.1. Игнорирование запроса с ошибочной CRC

Первое свойство требует, чтобы кадр с неверной CRC не приводил ни к записи, ни к ответу. Для проверки вводится макрос `finished`. Он истинен, когда Master получил ответ, зафиксировал тайм-аут или завершил широковещательный обмен. Макрос `no_writes` проверяет отсутствие операций записи у всех устройств. `unchanged(s)` сравнивает оба регистра одного Slave со снимком, а `all_unchanged` распространяет сравнение на все три устройства.

Листинг 2.10. Признаки завершения и сохранности памяти

```
#define finished ( \
    master_state == MASTER_RESPONSE || \
    master_state == MASTER_TIMEOUT || \
    master_state == MASTER_BROADCAST_DONE)

#define no_writes ( \
    !slave_write_performed[1] && \
    !slave_write_performed[2] && \
    !slave_write_performed[3])

#define unchanged(s) ( \
    holding_register_0[s] == previous_register_0[s] && \
    holding_register_1[s] == previous_register_1[s])

#define all_unchanged ( \
    unchanged(1) && unchanged(2) && unchanged(3))
```

Листинг 2.11. Свойство 1 Обработка ошибочной CRC

```
ltl p01_crc_ignored {
    [] ((finished && !crc_ok) ->
        (responses_sent == 0 && no_writes && all_unchanged &&
            (
                (slave_address == 0 &&
                    master_state == MASTER_BROADCAST_DONE) ||
                (slave_address != 0 &&
                    master_state == MASTER_TIMEOUT)
            )))
}
```

Формула проверяется во всех состояниях выполнения. Если обмен завершен и CRC неверна, число ответов должно быть равно нулю, флаги записи должны оставаться сброшенными, а память должна совпадать со снимком. Для адреса 0 ожидается завершение широковещательного обмена, для любого другого адреса ожидается тайм-аут.

Предпосылка не ограничивает функцию и адрес регистра. Поэтому требование действует и при сочетании неверной CRC с другими ошибками запроса.

2.4.2. Обращение к отсутствующему устройству

Второе свойство относится к адресу 4, для которого процесс Slave не создан. Такой запрос должен завершаться тайм-аутом, не затрагивая память существующих устройств и не создавая ответов.

Листинг 2.12. Свойство 2 Отсутствующий получатель

```
ltl p02_absent_slave {
    [] ((finished && slave_address == 4) ->
        (master_state == MASTER_TIMEOUT &&
            responses_sent == 0 && no_writes && all_unchanged))
}
```

Проверка не ограничивается корректными кадрами: при любом запросе к отсутствующему адресу результат должен удовлетворять указанным условиям. Это позволяет отдельно проверить фильтрацию получателя всеми Slave.

2.4.3. Отсутствие ответа на широковещательный запрос

Широковещательный запрос не должен создавать ответ даже во время обработки. Для этого используется `active`: он истинен в состоянии ожидания и в любом итоговом состоянии транзакции. Поэтому запрет ответа действует как во время обработки запроса, так и после ее завершения. Подготовка следующего запроса в `active` не входит: на этом этапе Master сбрасывает признаки предыдущего обмена и выбирает новые параметры.

Листинг 2.13. Признак активной транзакции

```
#define active (master_state == MASTER_WAITING || finished)
```

Листинг 2.14. Свойство 3 Молчание при широковещательной передаче

```
ltl p03_broadcast_silent {
    [] ((active && slave_address == 0) ->
        (responses_sent == 0 && len(slave_responses) == 0))
}
```

Условие проверяет одновременно счетчик отправленных ответов и длину канала. Макрос `active` позволяет обнаружить нарушение еще во время обработки запроса. После завершения обмена проверка с `finished` также выявила бы нарушение по ненулевому значению счетчика `responses_sent`.

2.4.4. Запись корректного широковещательного запроса

Для проверки записи вводится `valid_write`. Он требует верной CRC, функции `FC06` и существующего регистра, но не проверяет адрес получателя. Макрос `written(s)` устанавливает, что нужный регистр содержит переданное значение, а второй регистр этого Slave не изменился.

Листинг 2.15. Корректная запись и ее результат

```
#define valid_write (crc_ok && function_code == FC06 && register_address <= 1)

#define written(s) ( \
    (register_address == 0 && \
        holding_register_0[s] == register_value && \
        holding_register_1[s] == previous_register_1[s]) || \
    (register_address == 1 && \
        holding_register_1[s] == register_value && \
        holding_register_0[s] == previous_register_0[s]))
```

Листинг 2.16. Свойство 4 Запись всеми Slave

```
ltl p04_broadcast_written {
    [] ((finished && slave_address == 0 && valid_write) ->
        (master_state == MASTER_BROADCAST_DONE &&
            slave_write_performed[1] &&
            slave_write_performed[2] &&
            slave_write_performed[3] &&
            written(1) && written(2) && written(3)))
}
```

После завершения корректного запроса с адресом 0 все три устройства должны иметь установленный флаг записи и ожидаемое содержимое памяти. Отдельная про-

верка флагов существенна, если переданное значение уже находилось в регистре до запроса. Итоговое состояние Master должно соответствовать широковещательному завершению.

Здесь используется `finished`, поскольку во время обработки запись могли выполнить только один или два Slave. Требовать записи у всех трех устройств можно после завершения обмена.

2.4.5. Выполнение корректной адресной записи

Макрос `addressed` обозначает адрес существующего устройства, то есть диапазон от 1 до 3. Пятое свойство требует, чтобы корректная запись такому устройству завершалась нормальным ответом и действительно выполнялась у выбранного Slave.

Листинг 2.17. Признак существующего адресного получателя

```
#define addressed (slave_address >= 1 && slave_address <= 3)
```

Листинг 2.18. Свойство 5 Успешная адресная запись

```
ltl p05_addressed_write {
    [] ((finished && addressed && valid_write) ->
        (master_state == MASTER_RESPONSE &&
            response_exception_code == 0 &&
            (
                (slave_address == 1 && slave_write_performed[1] && written(1)) ||
                (slave_address == 2 && slave_write_performed[2] && written(2)) ||
                (slave_address == 3 && slave_write_performed[3] && written(3))
            )))
}
```

Три ветви соответствуют устройствам с адресами 1, 2 и 3. Для получателя запроса проверяется, что запись выполнена. Макрос `written(s)` проверяет, что нужный регистр содержит переданное значение, а второй регистр не изменился. Поля ответа и сохранность памяти остальных устройств проверяются отдельными свойствами.

2.4.6. Изоляция записи

Адресная операция должна изменять только выбранный регистр выбранного устройства. Макрос `isolated(s)` рассматривает два случая. Если Slave не является получателем, у него не должно быть ни операции записи, ни изменения памяти. Если запрос адресован ему, неизменным должен оставаться другой регистр. При ошибочном адресе регистра оба регистра должны оставаться неизменными.

Листинг 2.19. Проверка изоляции одного Slave

```
#define isolated(s) ( \
    (slave_address == s || (!slave_write_performed[s] && unchanged(s))) && \
    (slave_address != s || \
        (register_address == 0 && holding_register_1[s] == previous_register_1[s]) || \
        (register_address == 1 && holding_register_0[s] == previous_register_0[s]) || \
        (register_address > 1 && unchanged(s)) \
    ))
```

Листинг 2.20. Свойство 6 Изоляция адресной записи

```
ltl p06_write_isolation {
    [] ((finished && addressed) ->
        (isolated(1) && isolated(2) && isolated(3)))
}
```

Формула применяется ко всем завершенным адресным запросам, включая ошибочные. Проверка выполняется для всех трех Slave. У получателя должен оставаться неизменным регистр, не выбранный для записи, а у остальных устройств должны сохраняться значения обоих регистров.

2.4.7. Содержимое нормального ответа

Седьмое свойство начинается с факта получения нормального ответа. В таком случае исходный запрос должен быть корректной адресной записью, а количество отправленных ответов должно равняться одному.

Листинг 2.21. Свойство 7 Поля нормального ответа

```
ltl p07_normal_response {
    [] ((master_state == MASTER_RESPONSE && response_exception_code == 0) ->
        (addressed && valid_write && responses_sent == 1 &&
            response_slave_address == slave_address &&
            response_function_code == FC06 &&
            response_register_address == register_address &&
            response_register_value == register_value))
}
```

Проверяется совпадение адреса Slave, адреса регистра и значения с запросом, а также код функции `FC06`. Таким образом, формула исключает нормальный ответ на поврежденный, широковещательный или неподдерживаемый запрос и проверяет содержимое уже принятого сообщения.

2.4.8. Исключение для неподдерживаемой функции

Для описания исключений используется `exception_response(code)`. Он проверяет состояние получения ответа, единственность отправки, совпадение адреса и установку старшего бита кода функции. Дополнительно проверяются нулевые поля регистра и данных и требуемый код исключения.

Листинг 2.22. Общий вид ответа с исключением

```
#define exception_response(code) ( \
    master_state == MASTER_RESPONSE && responses_sent == 1 && \
    response_slave_address == slave_address && \
    response_function_code == function_code + 128 && \
    response_register_address == 0 && response_register_value == 0 && \
    response_exception_code == code)
```

Листинг 2.23. Свойство 8 Неподдерживаемая функция

```
ltl p08_illegal_function {
    [] ((finished && addressed && crc_ok && function_code != FC06) ->
        (exception_response(1) && no_writes && all_unchanged))
}
```

Если адресный запрос имеет верную CRC, но его функция отличается от FC06, после завершения требуется исключение 1. Операций записи быть не должно, все регистры должны совпадать со снимком. В генерируемых запросах неподдерживаемая функция представлена кодом FC03. Если одновременно указан несуществующий регистр, ответ также должен содержать исключение 1. Тем самым проверяется приоритет ошибки функции перед ошибкой адреса регистра.

2.4.9. Исключение для несуществующего регистра

Девятое свойство использует тот же макрос ответа, но требует код исключения 2. Предпосылка включает верную CRC и поддерживаемую функцию, поэтому проверяется именно ошибка адреса регистра.

Листинг 2.24. Свойство 9 Ошибочный адрес регистра

```
ltl p09_illegal_register {
    [] (
        (finished && addressed && crc_ok &&
            function_code == FC06 && register_address > 1) ->
        (exception_response(2) && no_writes && all_unchanged)
    )
}
```

После такого запроса ответ должен иметь правильное оформление исключения, а память всех Slave должна остаться неизменной. Флаги записи позволяют проверить также отсутствие попытки записи без видимого изменения значения.

2.4.10. Допустимость каждой выполненной записи

Макрос authorized(s) связывает установленный флаг записи с корректностью запроса и его адресом. Если Slave выполнял запись, запрос должен удовлетворять valid_write и быть адресован этому устройству либо всем устройствам сразу.

Листинг 2.25. Условие допустимости записи

```
#define authorized(s) ( \
    !slave_write_performed[s] || \
    (valid_write && (slave_address == 0 || slave_address == s)))
```

Листинг 2.26. Свойство 10 Запись только по допустимому запросу

```
ltl p10_write_authorized {
    [] (active -> (authorized(1) && authorized(2) && authorized(3)))
}
```

Условие проверяется во всех активных состояниях, поэтому нарушение можно обнаружить еще во время обработки запроса. Название authorized здесь означает допустимость операции по правилам модели: отдельный механизм аутентификации участников в нее не вводится.

2.4.11. Единственность ответа

Один запрос не должен приводить к нескольким ответам. Для проверки используется накопительный счетчик responses_sent, который сбрасывается только при подготовке следующей транзакции.

Листинг 2.27. Свойство 11 Не более одного ответа

```
ltl p11_single_response {
    [] (active -> (responses_sent <= 1))
}
```

Условие допускает ноль ответов, что необходимо для широковещательной передачи, неверной CRC и отсутствующего получателя. Одновременно оно запрещает повторную отправку в любой момент активной транзакции.

2.4.12. Обязательное завершение начатой транзакции

Свойства с условием `finished` определяют, каким должен быть результат завершенного обмена. Если Master бесконечно остается в `MASTER_WAITING`, условие `finished` не наступает, поэтому такие формулы сами по себе не обнаруживают зависание. Двенадцатое свойство отдельно требует завершения начатой транзакции.

Листинг 2.28. Свойство 12 Прогресс транзакции

```
ltl p12_transaction_progress {
    [] ((master_state == MASTER_WAITING) ->
        ((master_state == MASTER_WAITING) U finished))
}
```

Сильный оператор `U` требует, чтобы `finished` обязательно наступил, а до этого Master оставался в состоянии ожидания. Таким образом, запрос нельзя бесконечно обрабатывать или заменить новым запросом до фиксации результата.

Каждый Slave обрабатывает запрос за конечное число шагов, отправляет уведомление о завершении и ждет следующего запроса. Пока нового запроса нет, этот процесс заблокирован. Master получает уведомления от всех трех устройств и фиксирует результат обмена.

2.4.13. Согласованность состояния завершения

Следующее свойство уточняет смысл `finished`. К моменту фиксации результата все обработчики должны завершить запрос, а сообщения текущей транзакции должны быть извлечены из каналов.

Листинг 2.29. Свойство 13 Завершение всех обработчиков и очистка каналов

```
ltl p13_completion_clean {
    [] (finished ->
        (slave_processed[1] &&
            slave_processed[2] &&
            slave_processed[3] &&
            len(slave_requests[0]) == 0 &&
            len(slave_requests[1]) == 0 &&
            len(slave_requests[2]) == 0 &&
            len(slave_done) == 0 && len(slave_responses) == 0))
}
```

Проверяются три признака `slave_processed`, три очереди запросов, канал служебных уведомлений и канал ответов. Это условие не требует обнуления регистров: полезные данные сохраняются между транзакциями, а очищается только обмен сообщениями.

2.4.14. Ошибочный широковещательный запрос

Последнее свойство объединяет требования к широковещательной передаче и обработке ошибок. Если адрес равен нулю, но `valid_write` не выполняется, обмен должен завершиться без ответа и изменения памяти.

Листинг 2.30. Свойство 14 Ошибочная широковещательная передача

```
ltl p14_invalid_broadcast {
    [] ((finished && slave_address == 0 && !valid_write) ->
        (master_state == MASTER_BROADCAST_DONE &&
            responses_sent == 0 && len(slave_responses) == 0 &&
            no_writes && all_unchanged))
}
```

Формула охватывает все предусмотренные генератором ошибки широковещательного запроса. Ни один Slave не должен выполнить запись, счетчик и канал ответов должны остаться пустыми, а Master должен зафиксировать `MASTER_BROADCAST_DONE`. Совместно с четвертым свойством это описывает оба исхода широковещательной обработки: запись корректного запроса и отсутствие изменений при ошибке.

2.5. Автоматизация проверки

Для запуска верификации используется скрипт `run.py` на языке Python. Он работает со стандартной библиотекой и объединяет подготовку файлов, вызов SPIN, компиляцию верификаторов и обработку результатов. Проверка запускается командой `python run.py` из корня проекта.

2.5.1. Подготовка файлов и запуск инструментов

Работа начинается в функции `main`. В каталоге `verification_results` создается отдельная папка `run_*`, чтобы артефакты нового запуска не перезаписывали предыдущие. Функция `prepare_files` копирует в него модель, свойства и конфигурацию, после чего читает пути к SPIN и GCC из `config.json`. Имена проверяемых свойств извлекаются из объявлений `ltl` регулярным выражением.

Листинг 2.31. Подготовка входных данных проверки

```
def prepare_files(folder: Path) -> tuple[Path, Path, list[str]]:
    for filename in ("modbus.pml", "properties.pml", "config.json"):
        shutil.copy2(ROOT / filename, folder / filename)
    config = json.loads((folder / "config.json").read_text(encoding="utf-8"))
    spin = Path(os.path.expandvars(config["spin"]))
    cc = Path(os.path.expandvars(config["cc"]))
    os.environ["PATH"] = str(cc.parent) + os.pathsep + os.environ.get("PATH", "")
    source = (folder / "properties.pml").read_text(encoding="utf-8")
    properties = re.findall(r"^ltl\s+(\w+)\s*\{", source, re.MULTILINE)
    if not properties:
        raise RuntimeError("В properties.pml не найдены LTL-свойства")
    return spin, cc, properties
```

Функция `run` выполняет внешнюю команду через `subprocess.run`, сохраняет ее полный вывод в отдельный лог и проверяет код завершения. Для каждой команды установлен лимит в 180 секунд. Ошибка запуска, ненулевой код или превышение времени не позволяют считать проверку успешной.

Далее `build_verifiers` сохраняет версии инструментов, вызывает `spin -a properties.pml` и дважды компилирует полученный `pan.c`. Один исполняемый файл предназначен для структурной проверки модели, второй используется для проверки LTL-свойств. Основные команды приведены ниже.

Листинг 2.32. Генерация и запуск верификаторов

```
spin -a properties.pml
gcc -O2 -DCOLLAPSE -DSAFETY -DNOCLAIM -o pan_safety.exe pan.c
gcc -O2 -DCOLLAPSE -o pan_ltl.exe pan.c
pan_safety.exe -m1000000 -w22 -q -b
pan_ltl.exe -a -N p01_crc_ignored -m1000000 -w22 -b
```

Структурная проверка выявляет ситуации, в которых ни один процесс не может выполнить следующий шаг, хотя работа системы не завершена. Например, Master может ожидать уведомление, которое ни один Slave уже не способен отправить. SPIN рассматривает такую остановку как недопустимое конечное состояние. Эта проверка нужна, чтобы отдельно выявить ошибки обмена сообщениями перед проверкой LTL-свойств.

При компиляции верификаторов используются следующие параметры [5]:

— `-DNOCLAIM` исключает автомат `never claim` из структурной проверки. Поэтому она выполняется независимо от выбранной LTL-формулы.

— `-DSAFETY` отключает поиск циклов и сокращает затраты на проверку. Для обнаружения недопустимого конечного состояния достаточно конечной последовательности шагов, которая к нему приводит.

— `-DCOLLAPSE` сжимает хранимые состояния без потери данных и уменьшает расход памяти. Проверка остается полной. Приближенный режим `BITSTATE`, который может пропустить часть состояний, не используется.

При запуске LTL-верификатора параметр `-N` выбирает формулу по имени, а `-a` включает поиск принимающих циклов. Такой цикл описывает бесконечное выполнение модели, нарушающее выбранную формулу, например бесконечное ожидание вместо завершения транзакции.

Максимальная глубина обхода задана как 1 000 000 шагов. Параметр `-b` делает превышение глубины ошибкой, а `-q` в структурной проверке требует пустых каналов в допустимом конечном состоянии. `-w22` задает начальный размер хеш-таблицы поиска.

2.5.2. Обработка результатов и контрпримеров

Сам по себе нулевой код завершения программы еще не является подтверждением свойства. Функция `check_search_output` читает число ошибок, проверяет отсутствие сообщений о нехватке ресурсов и убеждается, что верификатор действительно выполнял полный поиск в нужном режиме.

Листинг 2.33. Проверка достоверности результата поиска

```
def check_search_output(name: str, output: str) -> int:
    errors = re.search(r"\berrors:\s*(\d+)", output)
    limits = (
        "max search depth too small", "depth limit reached", "out of memory",
        "cannot allocate", "vectorsz is too small", "time limit reached",
    )
    if not errors or any(text in output.lower() for text in limits):
        raise RuntimeError("Нет полного результата или достигнут предел ресурсов")
    if "Full statespace search for:" not in output:
        raise RuntimeError("SPIN не подтвердил полный поиск состояний")
    if name == "structure":
        if not re.search(r"invalid end states\s+\+", output):
            raise RuntimeError("Проверка конечных состояний отключена")
    else:
        if not re.search(r"never claim\s+\+", output):
            raise RuntimeError("LTL-проверка отключена")
        if not re.search(r"acceptance\s+cycles\s+\+", output):
            raise RuntimeError("Поиск принимающих циклов отключён")
    return int(errors.group(1))
```

Для структурной проверки должен быть включен поиск недопустимых конечных состояний. Для LTL должны быть активны `never claim` и поиск принимающих циклов. Затем функция `verify` различает найденное нарушение, успешную проверку и техническую ошибку.

Листинг 2.34. Формирование статуса одной проверки

```
def verify(name: str, command: list[str], folder: Path, spin: Path) -> tuple[str, str]:
    try:
        output = run(command, folder, name + ".log")
        error_count = check_search_output(name, output)
        if error_count > 0:
            save_counterexample(name, folder, spin)
            return "FAIL", f"Найден контрпример: {name}_trace.log"
        if "search not completed" in output.lower():
            raise RuntimeError("Поиск не завершён")
        states = re.search(r"([\d.e+]+) states, stored", output)
        depth = re.search(r"depth reached (\d+)", output)
        if not states or not depth:
            raise RuntimeError("Не удалось прочитать статистику поиска")
        return "PASS", f"состояний {states.group(1)}, глубина {depth.group(1)}, ошибок 0"
    except (OSError, RuntimeError) as error:
        return "ERROR", f"{error}; см. {name}.log"
```

Если обнаружено нарушение, `save_counterexample` сохраняет `properties.pml.trail` под именем свойства и запускает SPIN с параметрами `-t -p -g -l -k`. В отдельный лог записывается пошаговое воспроизведение с состояниями процессов и переменных. Если поиск не завершился или возникла ошибка инструмента, результат получает статус `ERROR`, а не `PASS`.

Функция `run_checks` сначала проверяет структуру модели, затем последовательно запускает все найденные LTL-свойства. В конце сохраняется `report.txt` с результатами и общим количеством `PASS`, `FAIL` и `ERROR`. Код завершения скрипта равен 0 при полном успехе, 1 при нарушении свойства и 2 при технической ошибке. При наличии и нарушения, и технической ошибки возвращается 2.

2.6. Верификация модели

Для верификации использовались SPIN 6.5.1 и GCC 16.2.0.

Сначала выполнялась структурная проверка без `never claim`, затем отдельно проверялась каждая из 14 формул. Такой порядок позволяет разделить недопустимое конечное состояние самой модели и нарушение конкретного требования.

В отличие от запуска одного сценария, верификация исследует допустимые варианты запросов и взаимодействия процессов. Память не сбрасывается между транзакциями, поэтому проверяются также последующие обращения к уже измененным регистрам. Начальное состояние при этом фиксировано: все регистры равны нулю, Master находится в состоянии подготовки.

2.6.1. Результаты проверки

Автоматический запуск выполнил 15 проверок и сформировал итог: `PASS=15, FAIL=0, ERROR=0`. Структурная проверка и все 14 LTL-свойств завершились без нарушений. Каждый поиск был выполнен полностью, ограничения времени и глубины не достигнуты. Контрпримеров не найдено. Результаты приведены в таблице 2.2.

Таблица 2.2. Результаты проверки модели

Проверка	Состояний	Глубина	Время, с	Итог
`structure`	22 782 712	179 270	20,9	PASS
`p01_crc_ignored`	22 782 712	310 296	28,2	PASS
`p02_absent_slave`	22 782 712	310 296	41,3	PASS
`p03_broadcast_silent`	22 782 712	310 296	48,4	PASS
`p04_broadcast_written`	22 782 712	310 296	71,8	PASS
`p05_addressed_write`	22 782 712	310 296	38,6	PASS
`p06_write_isolation`	22 782 712	310 296	29,5	PASS
`p07_normal_response`	22 782 712	310 296	27,2	PASS
`p08_illegal_function`	22 782 712	310 296	22,5	PASS
`p09_illegal_register`	22 782 712	310 296	22,9	PASS
`p10_write_authorized`	22 782 712	310 296	21,7	PASS
`p11_single_response`	22 782 712	310 296	24,4	PASS
`p12_transaction_progress`	45 128 020	310 296	66,1	PASS
`p13_completion_clean`	22 782 712	310 296	22,5	PASS
`p14_invalid_broadcast`	22 782 712	310 296	23,0	PASS

В таблице указано число сохраненных состояний из поля `states, stored` и время, сообщаемое самим SPIN. При LTL-проверке исследуется поведение модели совместно с автоматом свойства, поэтому число состояний может различаться между формулами. Достигнутая глубина оказалась меньше заданного предела в 1 000 000 шагов.

Проверка завершения транзакции потребовала почти вдвое больше памяти, чем остальные LTL-проверки: около 3 303 Мбайт. Суммарно все 15 поисков заняли 8 минут 29 секунд без учета генерации и компиляции верификаторов.

В журнале симуляции встретились все 120 сочетаний параметров запроса, включая варианты с одновременными ошибками CRC, кода функции и адреса регистра.

Результаты сохранены в папке `run_yuq4c6v_` каталога `verification_results`. Вместе с отчетом находятся итог скрипта `verification/automated_report.txt`, лог

свойства прогресса `verification/p12_transaction_progress.log` и сводная статистика `verification/summary.json`.

2.6.2. Существенные уточнения модели и свойств

Для проверки представленной версии изменять алгоритм Master и Slave или переписывать LTL-формулы не потребовалось. Все проверки выполняются одним запуском скрипта.

Перед каждым запросом модель сохраняет значения регистров, чтобы затем проверить их изменения. Флаги записи показывают, выполнялась ли запись, даже если в регистр записали то же значение. Счетчик ответов учитывает все отправленные ответы, включая уже прочитанные Master.

Master фиксирует результат после уведомлений от всех трех Slave, поэтому не объявляет тайм-аут до конца обработки. Условия `active` и `finished` позволяют проверять свойства во время обработки и после ее завершения. При подготовке следующего запроса они ложны, и эти проверки не выполняются.

Все выбранные свойства подтвердились для заданных параметров и последовательной обработки запросов. Для проверки всего Modbus-устройства нужна более полная модель.

3. Выводы

В рамках лабораторной работы рассмотрена модель обмена Modbus с одним ведущим и тремя ведомыми устройствами. На языке Promela описаны адресная и широковещательная запись регистра, проверка получателя, обработка неверной CRC, неподдерживаемой функции, ошибочного адреса регистра и сочетаний этих ошибок. Для модели сформулированы и разобраны 14 LTL-свойств, проверяющих сохранность памяти, корректность ответов и завершение обмена.

По результатам законченных поисков все свойства подтверждены, недопустимых конечных состояний не обнаружено. Все 15 проверок завершились успешно в одном автоматическом запуске. Ограничения ресурсов не достигнуты, дополнительная доработка модели и формул для проведения проверки не потребовалась.

Сохраненные значения регистров, флаги записи и счетчик ответов позволили проверить, как обработан каждый запрос. По уведомлениям от Slave Master определял, что обработка завершена. Поэтому при построении модели важно предусмотреть данные для проверки свойств и определить, в какой момент их проверять.

Рассмотренная модель описывает только небольшой фрагмент поведения Modbus. Даже при трех Slave, двух регистрах и ограниченном наборе значений большинство проверок сохраняло более 22 миллионов состояний, а проверка прогресса сохраняла более 45 миллионов. Успешные поиски суммарно заняли 8 минут 29 секунд и потребовали до 3 303 Мбайт памяти по отчету SPIN. При расширении состава устройств, диапазонов данных и правил обмена число сочетаний состояний и допустимых взаимодействий будет возрастать. Поэтому верификация более полной системы потребует дальнейшего выбора абстракций и, возможно, раздельной проверки ее частей. Полученный результат показывает корректность выбранного фрагмента в пределах явно заданных предположений.

Список использованных источников

[1] DafterT. Modbus Promela: репозиторий исходного кода [Электронный ресурс]. — GitHub. — URL: https://github.com/DafterT/modbus-promela (дата обращения: 28.10.2026).

[2] Modbus Organization. MODBUS Application Protocol Specification V1.1b3 [Электронный ресурс]. — URL: https://www.modbus.org/file/secure/modbusprotocolspecification.pdf (дата обращения: 28.10.2026).

[3] SPIN. Promela Reference: atomic [Электронный ресурс]. — URL: https://spinroot.com/spin/Man/atomic.html (дата обращения: 28.10.2026).

[4] SPIN. Promela Reference: LTL [Электронный ресурс]. — URL: https://spinroot.com/spin/Man/ltl.html (дата обращения: 28.10.2026).

[5] SPIN. Pan: The Verification Engine [Электронный ресурс]. — URL: https://spinroot.com/spin/Man/Pan.html (дата обращения: 28.10.2026).